

Case Study 02 - Designing a Cloud-native Architecture for an Online Course Platform

Saint Louis University

Professor: Greg Langston

IS-5500-23 Advanced Software Development

Harshitha Sama

September 16th, 2025

This particular case study architected a cloud-native design for an online course platform aimed at preserving security, scalability, and reliability while supporting high enrollment traffic. The solution AWS with a serverless-first design for APIs and with containerized workloads for sustained processes. Critical elements include high availability with auto-scaling, observability, and efficient costs.

Why AWS? (Choice Criteria)

Due to a comprehensive set of services AWS is financially beneficial considering the serverless Lambda, API Gateway, and Step Functions services. The choice of cloud provider is Amazon Web Services (AWS) due to its ecosystem, scalability, rich serverless features, and affordable price models.

Strengths of AWS:

- Breadth of services: Compute, databases, storage, analytics, and ML.
- Ripe No Server Stack: Lambda, API Gateway also with Step Functions.
- Global delivery: Must be done with CloudFront edge network.
- Security and compliance: IAM, KMS, WAF, and Shield.
- Pricing flexibility: Pay-as-you-go, reserved, and savings plans.
- Ecosystem and Documentation: In developer tooling and documentation, AWS shall pass GCP and Azure (Patil, 2024).

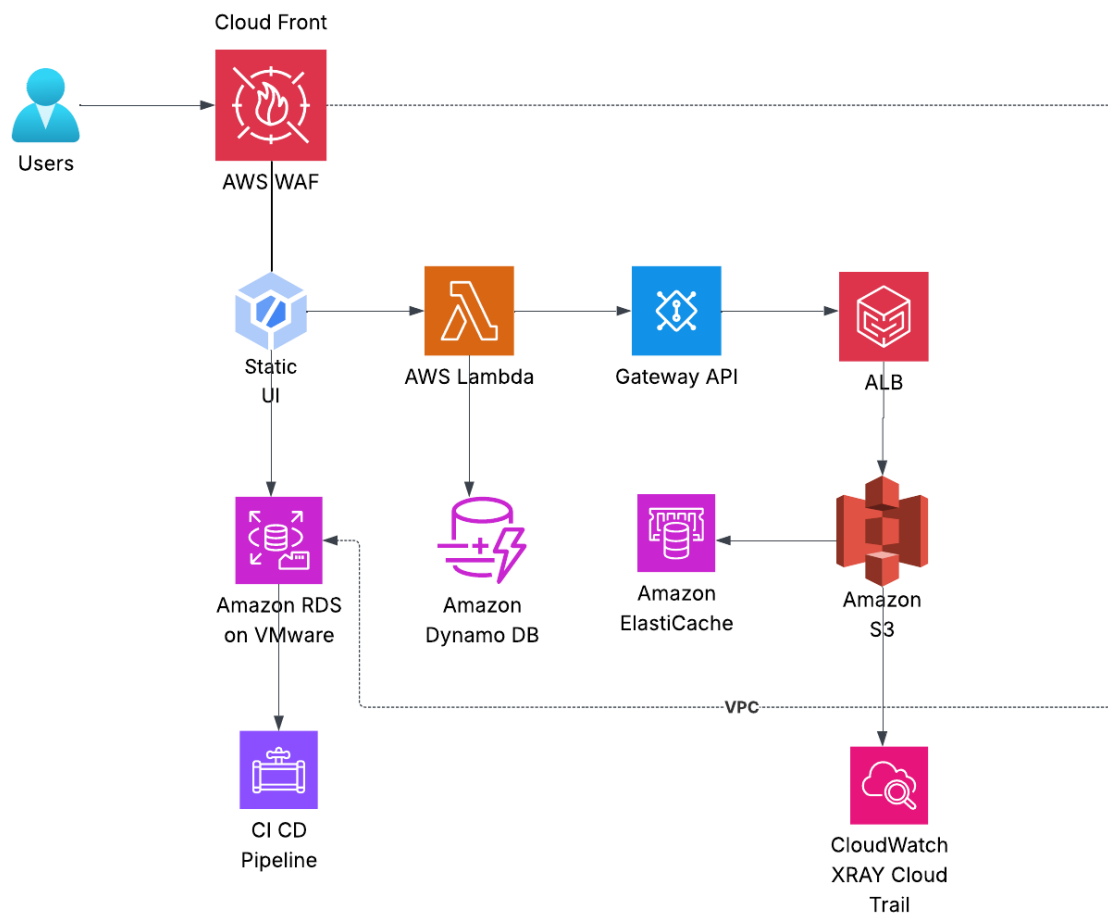
Comparison:

GCP: Data and ML services are excellent, but AWS is more widely adopted and richer.

Azure: AWS is the global leader with serverless offerings and the strongest with Windows workloads.

Trade-off: Vendor lock-in risk. Mitigation is containerized business logic and APIs.

High-level Architecture



The Web App is fronted by a React application sitting on S3 + CloudFront

Backed by Lambda and API Gateway for Backend APIs.

Data is stored in

- Relational data in Amazon RDS (PostgreSQL)
- And NoSQL quick access is Amazon DynamoDB.
- Course media is stored in Amazon S3.

Logic App: Serverless deployment is achieved via AWS Lamb and may be optionally

Containerized via ECS Fargate

Smoothing distribution is achieved via Application load balancing (ALB)

Redis (with ElastiCache) improves overall system efficiency by increasing throughput and reducing load on DB via faster access to cache on clustered nodes.

Control: Limit access by use of AWS WAF, and automatic application of IAM Roles, custodial VPC silos, Security Groups and NAT Gateway along with WAF.

Ensuring High Availability & Scalability

- ECS or EC2 instances are kept dynamically ready to spin down and spin up as required with the help of Auto Scaling.
- AWS Lambda functions provision themselves and scale automatically without any manual control to expand access.
- CloudFront and ALB both globally host incoming requests. They also facilitate load distribution and balancing (Dynamic Scaling for Amazon EC2 Auto Scaling - Amazon EC2 Auto Scaling, n.d.).
- RDS and S3 multi-AZ deployments have several fail-safes with huge durability at 99.999999999%. (Dynamic Scaling for Amazon EC2 Auto Scaling - Amazon EC2 Auto Scaling, n.d.).

Security Features

- Network isolation: Databases in isolated private subnets; NACLs and security groups restrict traffic.
- Identity & access: Fine-grained IAM roles, short-life credentials.
- Encryption: TLS in transit, KMS keys for data at rest in RDS/S3.
- AWS WAF and Shield: Protection against DDoS and general exploits.
- NAT Gateway and Private Subnets: Backend services are not given direct internet access.
- Logging and Auditing: Cloudtrail stores records of all API calls for compliance audits (Wiseman et al., 2018).

Monitoring & Logs

- CloudWatch: Metrics Collection, Alarms, and Dashboards.

- CloudWatch System Logs and Insights: Centralized logging capable of supporting queries.
- X-Ray: AWS traces microservice cross-interactions with transactions.
- CloudTrail: Security and account-related events are recorded at an account-level basis.
- Health Checks: Automated constant verification of the Services provided by the ALB to move forward traffic to the right Services.

Trade-offs and Challenges

Vendor Lock-in: AWS services dependency will always be a problem during migration.

Mitigation: Containerization and API documentation.

Cost forecast increases: no Service architecture horizontal scaling will be a problem under constant load.

Mitigation: Use Reserved capacity, Caching, and Budget ceilings.

Latency and cold starts: delays with with Lambda during flows sensitive and of lower latency.

Mitigation: Provisioned Concurrency or ECS for critical paths.

Operational Complexity: Microservices increases the debugging overhead.

Mitigation: Increased observability and standardized logs.

Data Consistency: Having ends with RDS, dynamo, and the cache sync issues.

Mitigation: Define and apply a single source of truth with optimistic locking.

Conclusion

This design is based on an AWS architecture. The design is cloud-native and provides a scalable, resilient, and secure online course platform. Usage of serverless APIs, containerized workloads, managed databases, and global edge delivery together bring an optimal cost-

performance balance. Operational excellence is delivered through security, observability, and CI/CD with the identified risks mitigated with design choices.

References

Dynamic scaling for Amazon EC2 Auto Scaling - Amazon EC2 Auto Scaling. (n.d.).

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scale-based-ondemand.html>

Patil, S. (2024, January 30). Cloud Showdown: AWS vs. Azure vs. Google Cloud - Choose

Wisely | CCSLA. CCSLA Learning Academy.

<https://www.ccslearningacademy.com/aws-vs-azure-vs-google-cloud/>

Wiseman, J., Plato, A., Anitian, & Anitian Corporation. (2018). PCI compliance in AWS. In

PCI Compliance in AWS.

https://d1.awsstatic.com/whitepapers/compliance/AWS_Anitian_Workbook_PCI_Cloud_Compliance.pdf